



BSc (Hons) in Information Technology Specialized in AI

**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

**Practical 01**

**Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

**Part 1: Code Analysis & PEAS Evaluation****Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

**Task 1.1: The Environment State (`_init_`)**

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure  
Environment  
Actuators  
Sensors

2. (Understand) Look at the `VisualGridHuntGame._init_` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width`  
`self.height`  
`self.walls`  
`self.food_positions`  
`self.opponents`  
`self.agent_pos`

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent

The environment contains multiple agents. Besides the main agent, there are opponent agents stored in `self.opponents`, which can move independently and interact with the main agent.

**Task 1.2: The Perception Subsystem (`get_percept`)**

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all percepts received by the agent from the beginning until the current time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors (S) component because it gathers information from the environment and provides it to the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This environment is Partially Observable because the agent does not have complete information about every aspect of the environment and must make decisions based on the percepts returned by `get_percept()`.

### Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure (P) is updated because the score is changed based on the agent's action.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

The performance measure should evaluate the agent's effect on the external environment, such as collecting food or hitting walls, because these actions show whether the goal is being achieved. Measuring internal processing effort only evaluates how the agent thinks, not whether it successfully performs its task. This prevents the agent from exploiting the performance measure, as shown in the Vacuum World example.

## Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

### Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes Partially Observable because the agent cannot perceive the locations of the toxic traps through its sensors.

### Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The `smells_toxin` sensor provides additional percepts about toxic traps. This helps the agent make better decisions by avoiding traps and choosing actions that maximize its performance measure. Therefore, the new sensor improves the agent's rationality.

### Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World exploit occurred when the performance measure rewarded the agent every time it cleaned dirt. The agent could repeatedly dump dirt and clean it again to gain unlimited points. This demonstrated that a performance measure should reward achieving the actual goal (keeping the environment clean) rather than rewarding individual actions.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING